

```
In [17]: # This code last updated on Feb 4, 2025

#Imports
import pandas as pd
import h5py
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, roc_curve, auc, precision_score, recall_score
import sklearn
import warnings
from sklearn.preprocessing import LabelEncoder
from sklearn.impute import KNNImputer
from sklearn.metrics import f1_score
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import cross_val_score
from sklearn.metrics import mean_squared_error, r2_score
from sklearn import svm
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import CountVecorizer
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import RobustScaler
from sklearn.preprocessing import MaxAbsScaler
from sklearn.preprocessing import Normalizer
from sklearn.preprocessing import QuantileTransformer
from sklearn.preprocessing import PowerTransformer
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
warnings.filterwarnings('ignore')

In [19]: #read in the file
hdfFile = h5py.File('passive_manualclassification_condensed2.h5', 'r')

#create dictionary to store the data
particleTable = {}

print('Reading in hdf5 file')

#List all keys in the file
hdfKeys = list(hdfFile.keys())
N = len(hdfKeys)

#allowedClassifications = [0,1,2] # now irrelevant like waterbeds
for i, key in enumerate(hdfKeys):
    dset = hdfFile[key]

    #check the 'classifications' attribute
    filteredClassifications = []    #you can make the values be unique by using a set() for this, instead of a list
    if 'classifications' in dset.attrs:
        classifications = dset.attrs['classifications']

        # valueCount = {0: 0, 1: 0, 2: 0}
        valueCount = [0,0,0]
        for user, value in classifications:
            if value == 2:
                valueCount[value] += 1

        # majority = len(classifications) // 2
        majority = np.argmax( valueCount )

        #check to see that there isn't a tie
        maxCount = 0
        for count in valueCount:
            if count == max( valueCount ): maxCount += 1
        if maxCount != 1:
            #there is a tie, that's not good
            continue

        for attr in dset.attrs:
            if attr not in particleTable:
                particleTable[attr] = []
            if attr == 'classifications':
                #we already filtered these, we handle this special
                particleTable[attr].append( majority )
            else:
                particleTable[attr].append(dset.attrs[attr])
        else:
            #there isn't a classification for this particle, WTF
            #probably we don't need this else because we're at the end of the loop
            continue

print(f"Filtered data has been loaded into particleTable with {len(particleTable)} attributes.")

Reading in hdf5 file
Filtered data has been loaded into particleTable with 31 attributes.

In [20]: #for key in particleTable:
#    print( key.ljust(15), len( particleTable[key]) )

print( 'The following classifications were issued' )
classificationCounts = {}
for value in particleTable['classifications']:
    if not value in classificationCounts:
        classificationCounts[value] = 0
    classificationCounts[value] += 1

classificationHeader = particleTable['classification_header'][0]
for key in classificationCounts:
    print( '%s %5i'%(classificationHeader[key], classificationCounts[key]) )

# for k in particleTable:
#     print ( k, len(particleTable[k]) )
print( len(hdfKeys) )

The following classifications were issued
s 10389
l 1916
b 2102
15008

In [21]: # Count how many times each user classified each classification

# make lists to store the number of times each user classified each type
count3195 = [0, 0, 0]
count3194 = [0, 0, 0]
count3225 = [0, 0, 0]
count3128 = [0, 0, 0]

for i, key in enumerate(hdfKeys):
    dset = hdfFile[key]
    if 'classifications' in dset.attrs:
        classifications = dset.attrs['classifications']
        for user, value in classifications:
            # if the value is allowed and fits a particular user, append that user's count list
            if value == 2:
                if user == 3195:
                    count3195[value] += 1
                elif user == 3194:
                    count3194[value] += 1
                elif user == 3225:
                    count3225[value] += 1
                elif user == 3128:
                    count3128[value] += 1

# calculate percentages for each user
#3195
snow3195 = count3195[0] / (count3195[0] + count3195[1] + count3195[2]) * 100
broken3195 = count3195[1] / (count3195[0] + count3195[1] + count3195[2]) * 100
ice3195 = count3195[2] / (count3195[0] + count3195[1] + count3195[2]) * 100
#3194
snow3194 = count3194[0] / (count3194[0] + count3194[1] + count3194[2]) * 100
broken3194 = count3194[1] / (count3194[0] + count3194[1] + count3194[2]) * 100
ice3194 = count3194[2] / (count3194[0] + count3194[1] + count3194[2]) * 100
#3225
snow3225 = count3225[0] / (count3225[0] + count3225[1] + count3225[2]) * 100
broken3225 = count3225[1] / (count3225[0] + count3225[1] + count3225[2]) * 100
ice3225 = count3225[2] / (count3225[0] + count3225[1] + count3225[2]) * 100
#3128
snow3128 = count3128[0] / (count3128[0] + count3128[1] + count3128[2]) * 100
broken3128 = count3128[1] / (count3128[0] + count3128[1] + count3128[2]) * 100
ice3128 = count3128[2] / (count3128[0] + count3128[1] + count3128[2]) * 100

print("Individual User Classifications:")
print(f"#3195: {count3195}")
print(f"#3194: {count3194}")
print(f"#3225: {count3225}")
print(f"#3128: {count3128}")
print(f"#recall that 0=snow, 1=broken, 2=ice (I think right? Double check with Mike or somethign because idk if I have i and b backwards)")
print(f"Percentages for 3195 = {snow3195:.2f}% s, {broken3195:.2f}% b, {ice3195:.2f}% i")
print(f"Percentages for 3194 = {snow3194:.2f}% s, {broken3194:.2f}% b, {ice3194:.2f}% i")
print(f"Percentages for 3225 = {snow3225:.2f}% s, {broken3225:.2f}% b, {ice3225:.2f}% i")
print(f"Percentages for 3128 = {snow3128:.2f}% s, {broken3128:.2f}% b, {ice3128:.2f}% i")

Individual User Classifications:
3195: [6950, 1310, 1545]
3194: [9897, 2691, 2028]
3225: [7520, 1230, 1871]
3128: [289, 196, 265]
Percentages for 3195 = 70.88% s, 13.36% b, 15.76% i
Percentages for 3194 = 67.71% s, 18.41% b, 13.88% i
Percentages for 3225 = 70.80% s, 11.58% b, 17.62% i
Percentages for 3128 = 38.53% s, 26.13% b, 35.33% i

In [22]: #This dataframe will contain all data essential to teaching the computer how to identify different classifications of precipitation
#It includes EVERY instance of a classification, so if a particular particle has three classifications,
#that situation will appear three times, one for each classification

df = pd.DataFrame()
for k in [ 'BrightHist1', 'BrightHist2', 'BrightHist3', 'BrightHist4', 'BrightHist5', 'BrightHist6', 'BrightHist7', 'BrightHist8', 'BrightHist9', 'BrightHist10',
          'Bright_avg', 'Bright_count', 'Bright_max', 'Bright_median', 'classifications', 'e', 'irreg', 'maj', 'maxIrreg', 'min', 'minIrreg', 'prt', 'r' ]:
    df[k] = particleTable[k]
df.dropna(1) #this will drop any rows that have a missing value -> prevent any issues with data holes
display(df)

BrightHist1 BrightHist2 BrightHist3 BrightHist4 BrightHist5 BrightHist6 BrightHist7 BrightHist8 BrightHist9 BrightHist10 ... Bright_median classifications e irreg maj maxIrreg
0 39.0 574.0 459.0 327.0 280.0 267.0 52.0 0.0 0.0 0.0 ... 0.2824 0 0.922 13.107 2.94 34.687
1 0.0 36.0 153.0 114.0 72.0 16.0 0.0 0.0 0.0 0.0 ... 0.3059 0 0.703 1.532 0.88 3.237
2 12.0 333.0 505.0 500.0 683.0 288.0 12.0 0.0 0.0 0.0 ... 0.3608 0 0.464 4.247 2.03 15.074
3 8.0 313.0 611.0 704.0 834.0 446.0 23.0 0.0 0.0 0.0 ... 0.3765 0 0.902 15.731 3.43 39.916
4 0.0 13.0 124.0 86.0 47.0 33.0 22.0 8.0 0.0 0.0 ... 0.3294 2 0.740 2.179 0.86 5.259
... ... ... ... ... ... ... ... ... ... ...
14402 0.0 1.0 24.0 49.0 35.0 30.0 5.0 0.0 0.0 0.0 ... 0.3922 2 0.552 0.812 0.49 1.423
14403 1.0 56.0 150.0 68.0 62.0 56.0 45.0 22.0 4.0 0.0 ... 0.3373 0 0.553 2.242 0.94 6.364
14404 1.0 84.0 164.0 142.0 67.0 28.0 2.0 0.0 0.0 0.0 ... 0.2980 0 0.645 1.55 0.95 3.371
14405 3.0 164.0 193.0 113.0 92.0 17.0 3.0 0.0 0.0 0.0 ... 0.2627 0 0.594 2.002 1.04 3.962
14406 0.0 3.0 68.0 55.0 65.0 38.0 4.0 0.0 0.0 0.0 ... 0.3765 2 0.547 1.006 0.63 2.324

14407 rows x 23 columns

In [23]: #ok, let's machine learn
#first, we split the data into two categories: the inputs and the target variable
X = df[['BrightHist1', 'BrightHist2', 'BrightHist3', 'BrightHist4', 'BrightHist5', 'BrightHist6', 'BrightHist7', 'BrightHist8', 'BrightHist9', 'BrightHist10',
        'Bright_avg', 'Bright_count', 'Bright_max', 'Bright_median', 'e', 'irreg', 'maj', 'maxIrreg', 'min', 'minIrreg', 'prt', 'r']]
y = df['classifications']

#in testing, I found that BrightHist2, 3, and 4 have "*****" string entries at some point, so we have to convert those to nan and drop them
#then, we make sure to match the y df to x so that they are the same length and play nice
X = X.apply(pd.to_numeric, errors='coerce')
X = X.dropna(1)
y = y[X.index]

# print(X.shape)
# print(y.shape)

# Check for non-numeric entries in X -> this is where I found that some columns had strings hidden in there a few times
# def check_non_numeric(df):
#     for column in df.columns:
#         if not pd.to_numeric(df[column], errors='coerce').notna().all():
#             print(f"Non-numeric values found in column: {column}")
# check_non_numeric(X)

#now, we split the data into the training set and the testing set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# normalize the datasets to improve the algorithms
# possible scalers/transformers: StandardScaler, MinMaxScaler, RobustScaler, MaxAbsScaler, Normalizer, QuantileTransformer, *PowerTransformer*
scaler = Normalizer()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

In [24]: ##### Logistic Regression Model #####
#make the model
model = LogisticRegression(random_state=42)
model.fit(X_train_scaled, y_train)

#make some predictions
LR_y_pred = model.predict(X_test_scaled)

#evaluate the model and print out some metrics/the results
accuracy = accuracy_score(y_test, LR_y_pred)
print("Accuracy = ", accuracy)

print("Classification Report: \n", classification_report(y_test, LR_y_pred))

print("Confusion Matrix: \n", confusion_matrix(y_test, LR_y_pred))

Accuracy = 0.7516521739130435
Classification Report:
              precision    recall  f1-score   support

     0       0.76      0.99      0.86      2132
     1       0.59      0.12      0.20       399
     2       0.36      0.03      0.06       344

 accuracy          0.57      0.38      0.75      2875
 macro avg          0.57      0.38      0.37      2875
weighted avg          0.69      0.75      0.67      2875

Confusion Matrix:
[[2101  26  5]
 [ 335  48 16]
 [ 324  8 12]]

In [25]: ##### Support Vector Machines #####
#create a SVM classifier
classifier = svm.SVC(kernel='rbf', gamma=0.1, random_state=42) # rbf very slightly beats out poly, but but are essentially tied for the best
#0.7329700272479565
#train the model using the training sets
classifier.fit(X_train_scaled, y_train)
#predict the response for test dataset
SVM_y_pred = classifier.predict(X_test_scaled)

# Model Accuracy: how often is the classifier correct?
print("Accuracy:", accuracy_score(y_test, SVM_y_pred))
# Model Precision: what percentage of positive tuples are labeled as such?
#print("Precision:", precision_score(y_test, SVM_y_pred, average = 'micro'))
# Model Recall: what percentage of positive tuples are labelled as such?
#print("Recall:", recall_score(y_test, SVM_y_pred, average = 'micro'))
print("Classification Report: \n", classification_report(y_test, SVM_y_pred))

print("Confusion Matrix: \n", confusion_matrix(y_test, SVM_y_pred))

Accuracy: 0.7415652173913043
Classification Report:
              precision    recall  f1-score   support

     0       0.74      1.00      0.85      2132
     1       0.00      0.00      0.00       399
     2       0.00      0.00      0.00       344

 accuracy          0.25      0.33      0.74      2875
 macro avg          0.55      0.74      0.63      2875
weighted avg          0.55      0.74      0.63      2875

Confusion Matrix:
[[2132  0  0]
 [ 399  0  0]
 [ 344  0  0]]

In [26]: ##### K Nearest Neighbors Model #####
knn_model = KNeighborsClassifier(n_neighbors = 32, weights = 'distance') # n_neighbors = 32 is the highest I found || random state doesn't work here
knn_model.fit(X_train_scaled, y_train)
#make predictions on the test data
knn_y_pred = knn_model.predict(X_test_scaled)

# Calculate the accuracy of the model
accuracy = accuracy_score(y_test, knn_y_pred)
print("Accuracy:", accuracy)
print("Classification Report: \n", classification_report(y_test, knn_y_pred))

print("Confusion Matrix: \n", confusion_matrix(y_test, knn_y_pred))

Accuracy: 0.7812173913043479
Classification Report:
              precision    recall  f1-score   support

     0       0.80      0.95      0.87      2132
     1       0.69      0.33      0.45       399
     2       0.53      0.23      0.32       344

 accuracy          0.67      0.51      0.78      2875
 macro avg          0.67      0.51      0.55      2875
weighted avg          0.75      0.78      0.75      2875

Confusion Matrix:
[[2034  44 54]
 [ 249 132 18]
 [ 250 14 80]]

In [27]: ##### Gaussian Naive Bayes #####
#we have to encode the classes to numeric so that the model runs
#this one works best with the Normalizer scaler
gnb = GaussianNB() #random state doesn't work here
# Train the classifier on the training data
gnb.fit(X_train_scaled, y_train)

# Make predictions on the testing data
gnb_y_pred = gnb.predict(X_test_scaled)

# Calculate the accuracy of the model
accuracy = accuracy_score(y_test, gnb_y_pred)
print("Accuracy: ", accuracy)

print("Classification Report: \n", classification_report(y_test, gnb_y_pred))

print("Confusion Matrix: \n", confusion_matrix(y_test, gnb_y_pred))

Accuracy: 0.7234782608695652
Classification Report:
              precision    recall  f1-score   support

     0       0.52      0.74      0.62      2132
     1       0.79      0.54      0.64       399
     2       0.38      0.74      0.50       344

 accuracy          0.61      0.70      0.72      2875
 macro avg          0.61      0.70      0.63      2875
weighted avg          0.60      0.72      0.75      2875

Confusion Matrix:
[[1573 205 354]
 [ 77 252 70]
 [ 64 25 255]]

In [28]: # automated random forest tuner - lowkey made it slightly worse
# param_grid = { "n_estimators" : [250, 300],
#               "criterion" : ["gini", "entropy"],
#               "max_features" : [3, 5],
#               "max_depth" : [10, 20],
#               "min_samples_split" : [2, 4] ,
#               "bootstrap" : [True, False] }
# random_search = RandomizedSearchCV(RandomForestClassifier(), param_grid)
# random_search.fit(X_train, y_train)
# print(random_search.best_estimator_)

In [29]: ##### Random Forest Model #####
# Create a Random Forest classifier
rf_classifier = RandomForestClassifier(n_estimators=140, random_state=42, bootstrap = True) #does bootstrap use replacement or no?
# Train the model on the training data
rf_classifier.fit(X_train_scaled, y_train)

# Make predictions on the testing data
rf_y_pred = rf_classifier.predict(X_test_scaled)

# Calculate accuracy
accuracy = accuracy_score(y_test, rf_y_pred)
print("Accuracy:", accuracy)

print("Classification Report: \n", classification_report(y_test, rf_y_pred))

print("Confusion Matrix: \n", confusion_matrix(y_test, rf_y_pred))

Accuracy: 0.84
Classification Report:
              precision    recall  f1-score   support

     0       0.87      0.94      0.90      2132
     1       0.79      0.54      0.64       399
     2       0.66      0.59      0.62       344

 accuracy          0.77      0.69      0.84      2875
 macro avg          0.77      0.69      0.72      2875
weighted avg          0.83      0.84      0.83      2875

Confusion Matrix:
[[1995  54  83]
 [1195 216 24]
 [ 138  2 204]]
```